

Presentación Quiz 3 – Dynamic Arrays and Amortized Analysis

Cristian Yecid Torres Rojas, Alfrek Arthur Pinto Garcia, Camilo Andres Arias Avilez
Universidad Nacional de Colombia, Sede Bogotá
Bogotá D.C., Colombia
{crtorresr, abadillop, cariasa}@unal.edu.co

EJERCICIO 1

(a) Implementación de *DynamicArray*

La estructura utiliza un arreglo genérico `data`, una variable `size` que representa el número de elementos almacenados y una variable `capacity` que representa la capacidad total. Cuando el arreglo se llena, su capacidad se duplica.

```
public class DynamicArray<T>{
    private T[] data; private int size,capacity;
    @SuppressWarnings("unchecked")
    public DynamicArray(int c){
        if(c<1)throw new IllegalArgumentException();
        capacity=c; size=0; data=(T[])new Object[c];
    }
    public int size(){return size;}
    public int capacity(){return capacity;}
    public T get(int i){check(i); return data[i];}
    public void set(int i,T v){check(i); data[i]=v;}
    public void append(T v){
        if(size==capacity)resize(2*capacity);
        data[size++]=v;
    }
    public T removeLast(){
        if(size==0)throw new IllegalStateException();
        T v=data[--size]; data[size]=null; return v;
    }
    private void check(int i){
        if(i<0||i>=size)throw new IndexOutOfBoundsException();
    }
    @SuppressWarnings("unchecked")
    private void resize(int c){
        T[] b=(T[])new Object[c];
        for(int i=0;i<size;i++)b[i]=data[i];
        data=b; capacity=c;
    }
}
```

(b) Invariantes de representación

Antes y después de cada operación pública deben cumplirse:

1. $0 \leq \text{size} \leq \text{capacity}$.
2. $\text{capacity} \geq 1$.
3. $\text{data.length} = \text{capacity}$.
4. Los elementos válidos se encuentran exactamente en $0 \leq i < \text{size}$.

(c) Complejidad en el peor caso

`get(index)` y `set(index,value)` acceden directamente a una celda, por lo que son $O(1)$. `removeLast()` también es $O(1)$ porque modifica únicamente la última posición válida. En `append(value)`, si existe espacio la inserción es $O(1)$, pero si $\text{size} = \text{capacity}$ se copian los n elementos anteriores. Por tanto, el peor caso de `append` es $O(n)$.

Operación	Peor caso
get	$O(1)$
set	$O(1)$
append	$O(n)$
removeLast	$O(1)$

(d) Costo amortizado de *append*

La política de crecimiento genera capacidades $1, 2, 4, 8, 16, \dots$. En una secuencia de n inserciones, los elementos copiados durante todos los redimensionamientos están acotados por

$$1 + 2 + 4 + \dots + \frac{n}{2} < n.$$

Las n escrituras ordinarias cuestan $O(n)$ y todas las copias también $O(n)$, luego $T(n) = O(n)$. El costo amortizado por inserción es

$$\frac{T(n)}{n} = \boxed{O(1)}.$$

Así, una inserción particular puede costar $O(n)$, pero el costo promedio sobre una secuencia es constante [1].

EJERCICIO 2

Se representa la serpiente como $[p_0, p_1, \dots, p_{k-1}]$, donde $p_0 = \text{TAIL}$ y $p_{k-1} = \text{HEAD}$. Un movimiento agrega una nueva cabeza mediante `append(newHead)` y, si la serpiente no come, elimina la cola desplazando el resto del arreglo.

(a) Complejidad de *REMOVE_TAIL()*

La operación equivalente es:

```
public void removeTail(){
    for(int i=1;i<size;i++) data[i-1]=data[i];
    data[size-1]=null;
    size--;
}
```

Para una serpiente de longitud k se realizan $k - 1$ copias: $p_1 \rightarrow p_0, p_2 \rightarrow p_1, \dots, p_{k-1} \rightarrow p_{k-2}$. Por tanto,

$$T_{\text{REMOVE_TAIL}}(k) = \Theta(k).$$

(b) *Complejidad total de m movimientos*

Cada movimiento sin comer ejecuta un **append**, de costo amortizado $O(1)$, seguido de **REMOVE_TAIL()**, de costo $\Theta(k)$. Entonces

$$T_{\text{MOVE}}(k) = O(1) + \Theta(k) = \Theta(k).$$

Si la longitud permanece constante en k durante m movimientos,

$$T(m, k) = m\Theta(k) = \boxed{\Theta(mk)}.$$

(c) *Costo amortizado de **MOVE()***

No es correcto concluir que **MOVE()** sea $O(1)$ amortizado solo porque **append()** lo sea. En m movimientos, todos los **append** cuestan $O(m)$, mientras que las eliminaciones cuestan $\Theta(mk)$. Así,

$$T_{\text{total}}(m, k) = O(m) + \Theta(mk) = \Theta(mk),$$

y al dividir por m se obtiene un costo amortizado $\boxed{\Theta(k)}$ por movimiento.

(d) *Bottleneck*

El cuello de botella es almacenar la cola físicamente en el índice 0. Eliminarla transforma $[p_0, p_1, \dots, p_{k-1}]$ en $[p_1, \dots, p_{k-1}]$ y obliga a desplazar $k - 1$ elementos en cada movimiento. El problema no es **append**, sino la representación lineal elegida.

EJERCICIO 3

Para evitar desplazamientos se usa un **arreglo circular**. Las variables de estado son **Position[] data**, **tail**, **head**, **size** y **capacity**. Para todo índice lógico $0 \leq i < \text{size}$ se define

$$f(i) = (\text{tail} + i) \text{ mód } \text{capacity}.$$

Así, $f(0) = \text{tail}$ y $f(\text{size} - 1) = \text{head}$. La aritmética modular permite “dar la vuelta” al final del arreglo sin mover todos los elementos.

(c,d) *Implementación en Java*

```
class Snake{
    private Position[] data;
    private int head,tail,size,capacity;
    Snake(int c){
        if(c<1)throw new IllegalArgumentException();
        capacity=c; data=new Position[c];
    }
    private int idx(int i){return (tail+i)%capacity;}
    Position get(int i){
        if(i<0||i>=size)throw new IndexOutOfBoundsException();
        return data[idx(i)];
    }
    void addHead(Position p){
        if(p==null)throw new IllegalArgumentException();
```

```
        if(size==capacity)resize();
        int h=(size==0)?0:(tail+size)%capacity;
        if(size==0)tail=0;
        head=h; data[head]=p; size++;
    }
    Position removeTail(){
        if(size==0)throw new java.util.NoSuchElementException();
        Position r=data[tail]; data[tail]=null;
        tail=(tail+1)%capacity; size--;
        if(size==0){tail=0;head=0;}
        return r;
    }
    private void resize(){
        int nc=2*capacity; Position[] b=new Position[nc];
        for(int i=0;i<size;i++)b[i]=get(i);
        data=b; capacity=nc; tail=0;
        head=(size==0)?0:size-1;
    }
}
```

(e) *Invariantes de representación*

1. $0 \leq \text{size} \leq \text{capacity}$, $\text{capacity} \geq 1$ y $\text{data.length}=\text{capacity}$.
2. $0 \leq \text{tail}, \text{head} < \text{capacity}$.
3. Cada elemento lógico i está exactamente en $f(i)$.
4. Las celdas que no pertenecen al intervalo circular ocupado permanecen en **null**.

(f) *Traza del arreglo circular*

La traza completa pedida en el enunciado, usando C, D, E, F, G, H como nuevas cabezas de N1–N6, es:

Paso	contenido físico	tail	head	size	cap.	resize
Inicio	[A,B]	0	1	2	2	–
N1 no come	[null,B,C,null]	1	2	2	4	sí
N2 come	[null,B,C,D]	1	3	3	4	no
N3 no come	[E,null,C,D]	2	0	3	4	no
N4 no come	[E,F,null,D]	3	1	3	4	no
N5 come	[E,F,G,D]	3	2	4	4	no
N6 no come	[null,E,F,G,H,null,null,null]	1	4	4	8	sí

N4 no redimensiona: se agrega F en el índice físico 1 y luego se elimina la cola C del índice 2. En N6, el **addHead** ocurre con **size=capacity=4**, por lo que primero se preserva el orden lógico $[D, E, F, G]$ al redimensionar a 8 y después se agrega H .

EJERCICIO 4

Sea m el número total de movimientos y $g \leq m$ el número de manzanas consumidas. La longitud estable inicia en 2 y termina en $2 + g$. Como **addHead()** ocurre antes de **removeTail()**, en un movimiento sin comer puede alcanzarse transitoriamente

$$L_{\text{máx}} \leq (2 + g) + 1 = g + 3.$$

(a,b) *Método agregado*

Con duplicación, un *resize* ocurre exactamente cuando el arreglo está lleno. Por ello las cantidades copiadas corresponden a capacidades previas reales

$$2, 4, 8, \dots, 2^r,$$

donde $2^r < L_{\text{máx}} \leq 2^{r+1}$. Así, el total de copias es

$$2 + 4 + \dots + 2^r = 2^{r+1} - 2 < 2L_{\text{máx}} = O(g + 1).$$

No basta con citar una serie geométrica: cada término aparece porque, justo antes de ese *resize*, **size=capacity** y deben copiarse todos los elementos de esa capacidad. En consecuencia, una secuencia de crecimientos hasta almacenar n elementos realiza $O(n)$ copias y tiene costo total lineal.

(c,d) Costo total y papel de m y g

Cada uno de los m movimientos ejecuta una escritura de nueva cabeza y trabajo aritmético $O(1)$; los $m - g$ movimientos sin comer además eliminan la cola en $O(1)$. Sumando las copias de los *resize*,

$$T(m, g) = O(m) + O(g + 1) = O(m), \quad g \leq m.$$

Por tanto, **addHead()** tiene costo amortizado $O(1)$. Los parámetros cumplen papeles distintos: m cuenta el trabajo ordinario de todos los movimientos, mientras g controla el crecimiento permanente y, por ello, cuántos umbrales de capacidad pueden alcanzarse. El aumento transitorio de $+1$ explica el término $g + 3$.

(e) Método potencial

Definimos, para tamaño s y capacidad C ,

$$\Phi(s, C) = \max\{0, 2s - C\} \geq 0.$$

Si **addHead()** no redimensiona, su costo real es 1 y $\Delta\Phi \leq 2$, de modo que el costo amortizado es a lo sumo 3. Si redimensiona, antes se tiene $s = C$ y después $s' = C + 1$, $C' = 2C$; por tanto

$$\hat{c} = (C+1) + \Phi(C+1, 2C) - \Phi(C, C) = (C+1) + 2 - C = 3.$$

En **removeTail()**, la capacidad no disminuye y el tamaño baja en uno, así que Φ no aumenta; su costo amortizado sigue siendo $O(1)$. Esto justifica explícitamente que las eliminaciones no destruyen la reserva necesaria para pagar futuros redimensionamientos.

EJERCICIO 5

Se comparan las políticas

$$A : C' = C + 1, \quad B : C' = 2C, \quad C : C' = \lceil 1.5C \rceil.$$

(a) Efecto del adversario

Si no existe contracción y la longitud máxima alcanzada es n , el momento exacto en que aparecen las manzanas no cambia las copias: cada política redimensiona únicamente al cruzar sus mismos umbrales de capacidad. Por tanto, para un mismo n se atraviesa la misma sucesión de capacidades y se copian los mismos tamaños, aunque los crecimientos estén separados por movimientos sin comer. Para aumentar el número de copias, el adversario necesitaría además poder **reducir la capacidad** (o forzar una política de contracción) y

después volver a provocar crecimiento, haciendo que se crucen repetidamente los mismos umbrales.

Política A: crecimiento aditivo

Cada vez que el arreglo está lleno se crea otro con una única celda extra y se copian todos los elementos anteriores. Para n inserciones,

$$\begin{aligned} T_A(n) &= \sum_{i=1}^n (i + 2) = \frac{n(n+1)}{2} + 2n \\ &= \frac{n^2 + 5n}{2} = O(n^2). \end{aligned}$$

El costo amortizado es entonces $O(n)$. El patrón inicial deja claro que se redimensiona en cada crecimiento:

i	e_i	size	capacity
0	0	2	2
1	3	3	3
2	4	4	4
3	5	5	5
4	6	6	6
5	7	7	7
6	8	8	8
7	9	9	9
8	10	10	10
9	11	11	11
10	12	12	12
11	13	13	13

Esta política minimiza memoria de reserva, pero hace que los *resize* sean demasiado frecuentes.

Política B: duplicación

Los redimensionamientos aparecen cuando la capacidad alcanza 2, 4, 8, 16, ... El costo de todas las copias forma una serie geométrica, de modo que

$$T_B(n) \leq 1 + 3n = O(n),$$

y el costo amortizado es $O(1)$. Los primeros crecimientos reproducen el patrón geométrico:

i	e_i	size	capacity
0	0	2	2
1	3	3	4
2	1	4	4
3	5	5	8
4	1	6	8
5	1	7	8
6	1	8	8
7	9	9	16
8	1	10	16
9	1	11	16
10	1	12	16
11	1	13	16

Política C: factor 1.5

Sea $c_0 = 2$ y

$$c_{j+1} = c_j + \left\lceil \frac{c_j}{2} \right\rceil.$$

Las capacidades generadas son 2, 3, 5, 8, 12, 18, 27, ... Si se requieren k redimensionamientos para alcanzar tamaño n , aproximadamente

$$n \leq 2(1.5)^k, \quad k \approx \lceil \log_{1.5}(n/2) \rceil.$$

El comportamiento inicial de la capacidad es:

i	e_i	size	capacity
0	0	2	2
1	3	3	3
2	4	4	5
3	1	5	5
4	6	6	8
5	1	7	8
6	1	8	8
7	9	9	12
8	1	10	12
9	1	11	12
10	1	12	12
11	1	13	18
12	1	14	18
13	1	15	18
14	1	16	18
15	1	17	18
16	1	18	18
17	19	19	27
18	1	20	27

La sucesión de capacidades 2, 3, 5, 8, 12, 18, 27, ... crece aproximadamente por un factor 1.5. Por ello, aunque no haya una expresión tan simple para cada e_i como en la política B, los eventos costosos siguen separados geométricamente.

Luego, usando la suma geométrica,

$$\begin{aligned}
T_C(n) &\approx n + \sum_{j=0}^k (1.5)^j \\
&= n + \frac{(1.5)^{k+1} - 1}{1.5 - 1} \\
&\leq n + 2(1.5)^2 \frac{n}{2} + 2 = O(n),
\end{aligned}$$

por lo que el costo amortizado también es $O(1)$.

Comparación de memoria y tiempo

Tras un *resize*, A deja 0 celdas libres; B deja aproximadamente $C_{\text{old}} = C_{\text{new}}/2$ (50%); y C deja aproximadamente $C_{\text{old}}/2 = C_{\text{new}}/3$ (33%). El resumen asintótico es:

Política	# resize	copias	amortizado	reserva
$C + 1$	$O(n)$	$O(n^2)$	$O(n)$	0 %
$2C$	$O(\log n)$	$O(n)$	$O(1)$	50 %
$\lceil 1.5C \rceil$	$O(\log n)$	$O(n)$	$O(1)$	33 %

A usa menos memoria extra, mientras que B y C reducen drásticamente la frecuencia de las operaciones costosas. En B, inmediatamente después de duplicar, aproximadamente la mitad del nuevo arreglo queda libre. En C, la ocupación es aproximadamente

$$\frac{C}{1.5C} = \frac{2}{3},$$

por lo que queda libre cerca de un tercio de la capacidad total.

Generalización para $\alpha > 1$

Considere la política

$$C' = \lceil \alpha C \rceil, \quad \alpha > 1 \text{ constante.}$$

Tras aproximadamente $k = \lceil \log_\alpha n \rceil$ redimensionamientos,

$$\begin{aligned}
T(n) &\approx n + c_0 \sum_{i=0}^k \alpha^i \\
&= n + c_0 \frac{\alpha^{k+1} - 1}{\alpha - 1}.
\end{aligned}$$

Como $k \leq \log_\alpha n + 1$, se tiene $\alpha^{k+1} \leq \alpha^2 n$. Así,

$$T(n) \leq n + c_0 \frac{\alpha^2 n - 1}{\alpha - 1} = O(n),$$

porque $\alpha > 1$ es una constante. Por tanto, cualquier política geométrica con factor fijo $\alpha > 1$ tiene costo amortizado $O(1)$. Además, el número total de elementos copiados es aproximadamente

$$\sum_{i=0}^k c_i \approx c_0 \frac{\alpha^{k+1} - 1}{\alpha - 1} \approx 2 \frac{\alpha n - 1}{\alpha - 1}.$$

Esto muestra el compromiso fundamental: un α mayor reduce el número de redimensionamientos y copias, pero reserva más memoria; un α más cercano a 1 conserva memoria a costa de redimensionar con mayor frecuencia.

REFERENCIAS

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
- [2] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, *Data Structures and Algorithms in Python*, Wiley, 2013.